

KubeCompute

Plataforma de procesamiento distribuido con workers efímeros gestionados por Kubernetes

Tópicos Avanzados en Informática · Maestrías en Ingeniería de Sistemas y Seguridad Digital
Pontificia Universidad Javeriana · Bogotá D.C., Colombia

El reto no es calcular: es orquestar

Ejecutar tareas intensivas en una sola máquina limita el escalamiento y complica la gestión manual de recursos.

Problema Central

Integración entre procesamiento distribuido, ciclo de vida de workers y orquestación de infraestructura.

- Determinación dinámica de workers necesarios.
- División y asignación de subtareas.
- Consolidación de resultados y liberación de recursos.

Durability Score: 4/5 — Problema de Workflow, no de Output.



Usuario inicial y veto de confianza

Beachhead: Académico

Estudiantes e investigadores que necesitan experimentar con procesamiento paralelo sin construir el ciclo de vida de los workers.

Escenario Potencial

Pequeños equipos de I+D que requieren ejecutar cargas paralelizables sobre infraestructura Kubernetes existente.

Veto de Confianza: Administrador del Clúster

El responsable de infraestructura es el decisor crítico. La adopción depende de garantizar que el sistema no consuma recursos de forma descontrolada ni interfiera con otras cargas de trabajo críticas.

Confianza como ventaja verificable

Ejecuciones Observables

Trazabilidad total: workers utilizados, subtareas ejecutadas y estados de cada fragmento de procesamiento.

Recursos Controlados

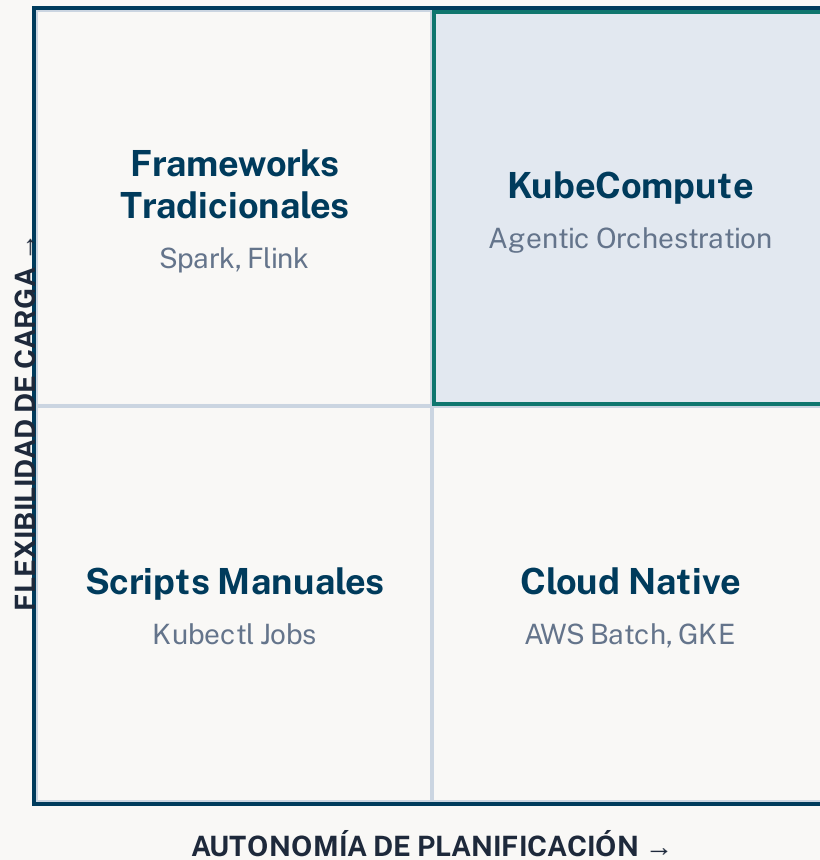
Límites estrictos de CPU y memoria por worker. Cuotas máximas de Pods para proteger la estabilidad del clúster.

Resultados Reproducibles

Capacidad de reconstruir ejecuciones y comparar resultados con referencias secuenciales para garantizar integridad.

"La confianza no es una promesa, es una propiedad verificable del sistema."

Complementar el ecosistema, no reemplazarlo



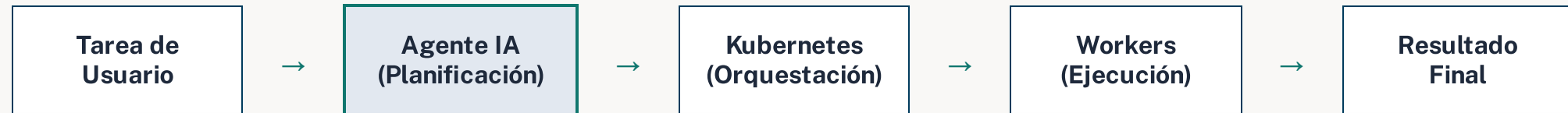
Enhancer (AI-Enhanced)

No compite con Kubernetes ni con Spark. Utiliza la infraestructura existente y añade una capa de inteligencia para la planificación operativa.

Diferenciación

Integración entre **Agente** → **Planificación** → **Ejecución**. La IA no es solo una interfaz, es el planificador que optimiza la estrategia de paralelización.

De una tarea a un resultado evaluado



1. Interpretar

Analiza la naturaleza de la tarea y datos.

2. Planificar

Selecciona estrategia y número de workers.

3. Ejecutar

Despliega Jobs y monitorea Pods.

4. Recuperar

Detecta fallos y reintenta dentro de límites.

5. Consolidar

Agrega resultados y evalúa el rendimiento.

La autonomía está limitada por políticas: máximo de workers, cuotas de CPU/Memoria y operaciones restringidas sobre el clúster.

La apuesta: velocidad total de procesamiento

Sacrificio de Costo

Uso de modelos de IA con mayor capacidad para asegurar una planificación robusta.

Sacrificio de Simplicidad

La capa agentic añade complejidad frente a ejecuciones determinísticas.

Enfoque en Speedup

El objetivo es reducir el tiempo total de la carga, no la latencia de respuesta del agente.

Prioridad Estratégica

SPEED

Optimización del tiempo de ejecución distribuida

Métricas Clave

Speedup (Sp): $T1 / Tp$

Eficiencia (Ep): Sp / p

Prototipo académico: medir antes de monetizar

CATEGORÍA	MÉTRICA DE ÉXITO	DEFINICIÓN OPERATIVA
Usuario	Tiempo total de ejecución	Comparación Secuencial vs Distribuida (Kubernetes).
IA / Agente	Precisión de Planificación	% de estrategias que cumplen criterios de rendimiento.
Técnica	Speedup y Eficiencia	Relación entre recursos invertidos y mejora de tiempo.
Operativa	Tasa de Completitud	% de tareas que finalizan con resultados válidos.

Modelo Económico Hipotético: Freemium / Reverse Trial basado en recursos consumidos (CPU/Memoria/Tiempo). Para el prototipo académico, el enfoque es la validación técnica y no la proyección comercial.

Riesgos críticos y mitigaciones

1. Comoditización

Frameworks existentes o nubes pueden integrar planificación IA rápidamente.

Mitigación: Foco en la integración experimental y observabilidad profunda, no solo en ejecución.

2. Sobreaprovisionamiento

El agente podría decidir usar recursos excesivos sin mejora proporcional en tiempo.

Mitigación: ResourceQuotas, límites estrictos por worker y registro de decisiones del agente.

3. Integridad del Resultado

Particionamiento incorrecto o fallos en la consolidación de subtareas.

Mitigación: Verificación independiente del resultado final y comparación con ejecución secuencial.

4. Replicabilidad

La arquitectura básica es replicable en poco tiempo por equipos expertos.

Mitigación: Moat basado en la calidad de la planificación y evidencia experimental acumulada.

Hipótesis de validación

"La capa agentic toma decisiones de planificación suficientemente útiles como para justificar su existencia frente a una configuración determinística y frameworks existentes."

Propuesta de Proyecto Final · Tópicos Avanzados en Informática
Preguntas y Discusión